CortexHR — AI/ML Developer Guide

WorkforceOS · Section 2 · The AI Intelligence Layer

One complete working pack for Tasks 1 to 20: what you receive, what you build, how every handoff works, and the exact answer to every common question.

---

1. Start here

This guide, together with **workbook 02 (02_CortexHR_AI_ML_Dataset.xlsx)**, **workbook 01 (01_LedgerForge_Python_Master_Dataset.xlsx)** and your original task brief, is everything you need for all 20 tasks. Workbook 01 is used only to build the small "mock backend" for the chatbot (Section 6).

Every task below has four short parts:

- **Inputs you already have**: the files, sheets and rules that task needs.
- **Decisions already made**: the choices that are normally argued about, settled for you.
- **Questions answered**: the doubts people usually have at that step.
- **Proof to submit**: what to attach to your pull request when the task is done.

If something is not clear

1. Look in the task block in this guide.
2. Look in the **README**, **data_dictionary** and **answer_key_notes** sheets of workbook 02 (open the answer-key sheets only after your own audit in Task 1).
3. Pick the simplest sensible option. Write one line in the **Assumptions** section of your README (what you assumed and why). Continue.

An assumption written in your README is accepted as it is. You can always continue working.

How your work is submitted

- Everything lives in **your own GitHub repository**: code, models, documents, reports.
- One pull request per task. The pull-request link is the "GitHub link" your reports ask for.
- Phase reviews and Reports 1, 2 and 3 are markdown files in `reports/` (templates in Section 10).

- Give your repository link and report links through the same submission channel your programme already uses for your work (for example GrowMar Connect).

---

2. Your starter kit

Item	What it is
Workbook 02	32 sheets: attrition training table, anomaly sets with hidden answers, salary and headcount history, resumes, chatbot data, reference scores, sample outputs, answer keys. Fake data only.
Workbook 01	Employees, leave, attendance and payslips for the mock backend only.
Software	Python 3.12, Docker Desktop (Compose v2), Git, VS Code, Jupyter, curl or Postman. All free.
Libraries (pin versions)	pandas, numpy, scikit-learn, xgboost, optuna (optional), shap, lime, fairlearn, prophet (or statsmodels), spaCy, sentence-transformers (optional), FastAPI, uvicorn, pydantic, PyJWT, joblib, httpx, prometheus-client, pytest, locust. Rasa 3.x lives in its own container (Python 3.10).
AI design tool	Any free tool (needed only in Task 19).

The company in the data is **Zenith Nova Technologies Pvt Ltd** (fictional): 745 employees ever employed, 441 active, 10 departments, history from January 2020 to **30 September 2026**.

---

3. Dependency register (what you need, and where it is)

What you need	Where it is	Needed in
Training data for every model	Workbook 02	Tasks 1 to 19
Employee, leave, attendance and payslip data for the chatbot	Workbook 01 through the mock backend in Section 6	Tasks 9, 19

What you need	Where it is	Needed in
Login and role rules	Section 5	Tasks 7, 11, 19
Output formats for risk, anomaly and forecast results	Workbook 02 sheets sample_attrition_risk, sample_anomaly_results, sample_forecast	Tasks 6, 11, 14, 16
Reference scores to compare yourself with	Workbook 02 sheets baseline_benchmarks and anomaly_baselines, and Section 7	Tasks 3, 4, 5, 12
Chatbot replies, intents and test messages	Workbook 02 chatbot sheets	Tasks 8, 9, 19
Deployment readiness checklist	Section 9	Tasks 15, 20
Hosting platform for the live link	Announced separately when final. Section 9 shows how to be fully ready before that.	Tasks 15, 20
Report and review templates	Section 10	All reports

4. Handoff register (what you publish, and how)

How a handoff works: you publish the item in your own repository at the path shown, link it from your README under a heading **Handoff package**, and mention it in your phase review. Other teams read it from your repository. There is nothing to email, upload or send anywhere else.

Item	When	Publish at	What it contains
Data audit and baseline report	End of Phase 1	`reports/data-audit.md`, `reports/baseline-report.md`	Findings and scores in plain language
Anomaly service guide	Task 6	`docs/ai-api-guide.md`	Endpoints with example requests and responses

Item	When	Publish at	What it contains
Login and model version rules	Task 7	`docs/auth-and-versions.md`	How tokens are checked, how to switch or roll back a model
Unified AI API guide	Task 11	`docs/ai-api-guide.md` (updated)	All endpoints, errors, examples
Chat screen and embed guide	Task 19	`chat.html`, `chat-widget.js`, `docs/handoff-fullstack.md`	How to mount the widget
Output formats for dashboards	Task 20	`docs/handoff-data.md`	Columns of risk, anomaly and forecast outputs
Health and metrics	Task 20	`docs/handoff-devops.md`	Health, metrics and alert rules
Security notes and fairness report	Task 20	`docs/handoff-security.md`, `reports/fairness-report.md`, `docs/model-cards/`	Auth use, logging rules, fairness results
Live AI service link	Tasks 15 and 20	README and report	URL plus health URL

---

5. Shared standards

Ports and variables (use exactly these)

Service	Local port	Base path
Your AI services	8100	`/ai/v1`
Rasa (if used)	5005	
Mock backend (Section 6)	8000	`/api/v1`

Variable	Local default
`ENV`	`local` (`production` when deployed)
`JWT_SECRET`	`zenith-nova-local-dev-secret-key-change-in-production` (local only)
`JWT_ALGORITHM`	`HS256`
`BACKEND_BASE_URL`	`http://localhost:8000/api/v1`
`DEMO_TODAY`	`2026-09-30` (use for "today" so your numbers match the data)
`ACTIVE_MODEL_<NAME>`	Version folder to serve, for example `ACTIVE_MODEL_ATTRITION=2.0.0`

Commit `.env.example` with these names. Never commit `.env`.

Tokens (your service only checks them)

- Token = JWT, HS256, signed with `JWT_SECRET`. Claims: `sub` (user id like U-0001), `employee_id`, `role` (ADMIN, HR or EMPLOYEE), `type` (`access`), `iat`, `exp`, `jti`. Header: `Authorization: Bearer <token>`.
- To test locally, make tokens yourself with `scripts/make_token.py`:

```
import jwt, time, sys
claims = {"sub": "U-0001", "employee_id": sys.argv[1], "role": sys.argv[2],
"type": "access",
          "iat": int(time.time()), "exp": int(time.time()) + 3600, "jti":
"local-test"}
print(jwt.encode(claims, "zenith-nova-local-dev-secret-key-change-in-
production", algorithm="HS256"))
```

Run `python scripts/make_token.py EMP-0100 HR` to get an HR token. Employee ids and roles come from the **users** sheet of workbook 01.

Conventions for every endpoint

- **Errors:** `{"error": {"code": "VALIDATION_ERROR", "message": "...", "details": [{"field": "...", "issue": "..."}], "request_id": "..."}}`. Codes: VALIDATION_ERROR 422, UNAUTHENTICATED 401, FORBIDDEN 403, NOT_FOUND

404, INTERNAL_ERROR 500. No stack traces.

- **Dates** ISO. **Money** as a string with 2 decimals in JSON.
- **Health:** `GET /ai/v1/health`, no login: `{"status": "ok", "service": "cortexhr", "version": "...", "checks": {"models": "ok", "chatbot": "ok", "backend": "ok"}}`.

Roles

What	ADMIN	HR	EMPLOYEE
Attrition risk, payroll flags, salary bands, forecasts, resume scores	yes	yes	no
Own attendance flags, leave balance, payslip status, manager, team	yes	yes	own data only
Models, metrics, drift endpoints	yes	no	no

---

6. The mock backend and the AI contract

Mock backend (build it once in `mocks/backend/`)

A tiny FastAPI app on port 8000 that reads workbook 01 and answers what the chatbot needs. It checks the same JWT as in Section 5 and treats “me” as the `employee_id` in the token. Sheets used: employees, employee_assignments, leave_balances, leave_requests, payroll_items, attendance.

Call	Returns (field names equal workbook 01 columns)
`GET /employees/me`	`employee_id, full_name, department_id, department_name, role_title, manager_id, manager_name`
`GET /employees/me/team`	list of `employee_id, full_name, role_title` for people whose `manager_id` is you

Call	Returns (field names equal workbook 01 columns)
`GET /leave/balance?employee=me`	`employee_id, as_of_date, financial_year, cl_balance, sl_balance, el_balance`
`GET /leave/requests?employee=me&limit=1`	`items: [{leave_request_id, leave_type_code, start_date, end_date, days, status}]`
`GET /payslips/{employee_id}/{month}` (month `2026-09`)	the payroll_items columns for that employee and month; an employee asking for someone else gets 403
`GET /attendance/summary?employee=me&month=2026-09`	`scheduled_days, present_days, late_arrivals, unpaid_absence_days, attendance_rate` (sum the attendance rows of the month; rate = (present + 0.5 x half days) / scheduled)

The real backend uses the same paths and field names. Switching from the mock to the real one is only a change of `BACKEND_BASE_URL`.

AI services contract (port 8100, base `/ai/v1`)

Endpoint	Purpose	Who may call
`POST /anomaly/attendance`	Score one or many employee-month attendance records	HR, ADMIN (own record for EMPLOYEE)
`POST /anomaly/payroll`	Score payroll rows	HR, ADMIN
`POST /attrition/score`	Attrition risk for employee ids or a department	HR, ADMIN
`GET /explain/attrition/{employee_id}`	Top drivers in plain language	HR, ADMIN

Endpoint	Purpose	Who may call
`POST /salary-band`	Low, median and high band for a role, level and experience	HR, ADMIN
`GET /forecast/headcount?department=D01&horizon=6`	Forecast with 80% range	HR, ADMIN
`POST /resume/score`	Score a resume against a job id	HR, ADMIN
`POST /chat`	Assistant: `{"message": "...", "session_id": "..."}`	any logged-in role (data rules per role)
`GET /models`, `GET /metrics` (Prometheus text), `GET /drift`, `GET /health`	Operations	`/health` open, others ADMIN

Result shapes follow the columns of the sheets **sample_attrition_risk**, **sample_anomaly_results** and **sample_forecast**. Those column names are the contract.

---

7. Reference numbers

Check	Value
attrition_dataset	2,389 rows (383 real employee snapshots + 2,000 extension rows + 6 duplicates); about 14.6% leave within 12 months
attrition_scoring_2026	430 active employees, same features, no label
attendance_monthly_features / payroll_monthly_features	7,339 / 7,355 rows with 150 / 100 injected problems (answers in the label sheets)
monthly_headcount_history	81 months x 11 departments + company total
salary_training	2,934 hires

Check	Value
Chatbot	858 training messages, 33 intents, 132 test messages, 33 reply templates, 30 FAQs, 12 dialogues
Honest model scores (5-fold, clean features)	Logistic Regression ROC-AUC about 0.71, Random Forest about 0.72, Decision Tree about 0.65, always-stays 0.50. A score near 1.0 means leakage.
Untuned Isolation Forest	about 59% recall and 40% precision on attendance anomalies; about 40% recall and 22% precision on payroll errors

Always true for this domain

- Use **stratified 5-fold cross-validation with seed 42** everywhere. Hold out a **20% stratified test set (random_state 42)** at Task 3 and do not touch it until Task 12.
- Never use gender or age as model inputs. They live in `protected_attributes` for testing only.
- Folder layout: `src/{features,models,services,chatbot,monitoring}`, `models_store/`, `reports/`, `mocks/`, `tests/`, `notebooks/` (exploration only).
- Python for the services is 3.12. **Rasa 3.x open source runs on Python 3.8 to 3.10, so give it its own container based on `python:3.10`.**

---

8. The 20 tasks

Phase 1 — Data foundations & baselines

Task 1 — Data audit & exploration

- **Inputs you already have:** sheet `attrition_dataset`, sheet `data_dictionary`.
- **Decisions already made:** write `reports/data-audit.md` with these sections: size and types, missing values, outliers (use IQR **and** z-score), class balance, duplicates, inconsistent category spellings, impossible values, leakage candidates (columns that exist only after someone leaves), and a **decisions table** (one row per problem: what

you will do and why). Default handling: drop exact duplicate rows, standardise category spellings, turn impossible values into missing, fill numbers with the median and add a “was missing” flag where more than 2% is missing.

- **Questions answered:** the sheets `answer_key_planted_dirt` and `answer_key_notes` exist. Finish your own audit first, then compare and add a “what I missed” section.
- **Proof to submit:** the report, a notebook that can be rerun, a Dockerfile that runs it.

Task 2 — Reusable feature pipeline

- **Inputs you already have:** attrition_dataset, attendance_monthly_features, payroll_monthly_features.
- **Decisions already made:** package `cortexhr_features`, version string `features-1.0.0` in `metadata.json`. Steps: the cleaning rules from Task 1, impute (median or most frequent), scale numbers, one-hot encode categories. Drop ids, dates, `source` and every leakage column. Engineered features: `tenure_years`, `raise_overdue` (months since last raise above 12), `unpaid_per_month`, `attendance_gap` (1 minus attendance rate), `overtime_load`. Separate transformers for attendance rows and payroll rows. Fit on training data only. Save with joblib. `python -m cortexhr_features.build` runs inside Docker.
- **Proof to submit:** tests (shape, no missing values after transform, same input gives same output).

Task 3 — Attrition baseline models & evaluation

- **Inputs you already have:** the pipeline, sheet baseline_benchmarks.
- **Decisions already made:** target `left_within_12m`. Models: always-stays dummy, Logistic Regression and Decision Tree (depth 4), both with balanced class weights. Report mean and spread over the 5 folds of ROC-AUC, PR-AUC, precision, recall and F1, plus the confusion matrix and ROC curve. Choose the threshold that gives **at least 60% recall** and report precision there. HR prefers catching leavers over avoiding false alarms; say so in the report.
- **Proof to submit:** `reports/baseline-report.md` in plain language.

Task 4 — Attendance anomaly detection

- **Inputs you already have:** attendance_monthly_features, attendance_anomaly_labels (answers), attendance_daily_examples.

- **Decisions already made:** Isolation Forest with 300 trees on unpaid-absence, streak, late, hours and rate features. Try contamination 1%, 2%, 3% and 5% and choose by the precision-recall trade-off. **People on long paid leave (maternity) are legitimate; use unpaid-based features so they are not flagged.** Use the labels **only to measure**, never to train. Beat the untuned reference with per-employee comparisons or simple rules. Save under `models_store/attendance_anomaly/1.0.0/` with `metadata.json` (threshold included).
- **Proof to submit:** a catch-rate and false-alarm table and the chosen threshold with a one-line reason.

Phase 1 review

Write `reports/phase-1-review.md` (template in Section 10).

Phase 2 — Attendance & payroll intelligence

Task 5 — Payroll outlier detection

- **Inputs you already have:** payroll_monthly_features, payroll_anomaly_labels (answers), and the **employees** sheet of workbook 01 (for the column `pf_applicable`).
- **Decisions already made:** build rule checks first (negative net pay, zero pay, overtime above 50 hours, duplicate employee-month, gross above 2.5 times monthly fixed gross with no variable pay, PF missing for a PF-applicable employee, gross well below fixed pay with no loss of pay), then an ML detector on ratio features, then the two together. Use these **reason codes**: `DUPLICATE_PAYMENT, NEGATIVE_NET_PAY, ZERO_PAY, OVERTIME_ABOVE_CAP, GROSS_ABOVE_NORM, PF_NOT_DEDUCTED, GROSS_BELOW_EXPECTED, STATISTICAL_OUTLIER`. **Target for rules plus ML: recall of at least 85% with precision of at least 50%.**
- **Questions answered:** use the labels only to measure, never to train an unsupervised detector.
- **Proof to submit:** a comparison table (rules, ML, both) and a reason code on every flag.

Weekly Milestone 1 (Tasks 1 to 5)

Build a small “Data Health Check” script that runs on any new export and prints missing values, new categories, out-of-range values and label balance. Then answer in two lines each: why 90% accuracy can still be useless, why scalers are fitted on training data only,

why anomalies are injected, and the difference between precision and recall in HR words. Save as `reports/weekly-1.md`.

Task 6 — Anomaly detection microservice

- **Inputs you already have:** Section 6 (contract), the two models from Tasks 4 and 5, sheet sample_anomaly_results (response shape).
- **Decisions already made:** FastAPI on port 8100, base `/ai/v1`. `POST /anomaly/attendance` and `POST /anomaly/payroll` accept one record or a list. Field names equal the columns of the feature sheets. Responses use the columns of sample_anomaly_results (`anomaly_score, flagged, reason_code, reason_text, model_version`). Load models once at start-up. Validate every field with Pydantic and return the error format of Section 5. Add `/health` and `/models`. Keep the Docker image slim.
- **Proof to submit:** tests for good input, bad input, an empty list and a missing model file; `docs/ai-api-guide.md` with one example call per endpoint.

Task 7 — Model serving with auth & versioning

- **Inputs you already have:** the token rules in Section 5 and the script `scripts/make_token.py`.
- **Decisions already made:** verify tokens with `JWT_SECRET`. HR and ADMIN may score anyone; EMPLOYEE may score only their own `employee_id`. Registry: `models_store/<name>/<version>/{model.joblib, metadata.json}` (date, data version, parameters, metrics). The active version is chosen by `ACTIVE_MODEL_<NAME>`; **a rollback is a change of that variable and a restart**. `GET /models` lists versions and metrics.
- **Proof to submit:** tests for no token, bad token, expired token, wrong role and a version switch; `docs/auth-and-versions.md`. Submit **Report 1** after this task.

Phase 2 review

Write `reports/phase-2-review.md` and **Report 1** (templates in Section 10).

Phase 3 — Chatbot & NLP

Task 8 — HR FAQ chatbot

- **Inputs you already have:** chatbot_nlu_training (858 messages, 33 intents), chatbot_test_set (132), chatbot_responses (33), hr_faq (30).
- **Decisions already made:** primary stack is Rasa 3.x in its own Python 3.10 container. **If Rasa is not working after half a day, use the fallback:** a scikit-learn or spaCy intent classifier plus your own small dialogue code, and write the reason in your README. Train only on the training sheet; evaluate on the test sheet. **Target: at least 85% intent accuracy.** A confidence below 0.55 returns the `out_of_scope` reply. The intent `salary_of_other` is always refused. Replies come from `chatbot_responses`.
- **Proof to submit:** accuracy and the list of wrongly classified test messages.

Task 9 — Chatbot with live data

- **Inputs you already have:** the mock backend in Section 6, column `backend_or_service_call` of chatbot_responses, sheet chatbot_dialogues.
- **Decisions already made:** live intents are leave_balance, leave_request_status, payslip_status, my_manager, my_department, team_members, attendance_summary and unusual_attendance. Read the base URL from `BACKEND_BASE_URL`. Always forward the user's own token; the backend decides what the user may see. Month words use `DEMO_TODAY`: "last month" means 2026-08; a month name from March to September means 2026; October to December means 2025. If the backend is down or answers 403, reply with the `on_failure` text; never crash and never guess a number. "unusual_attendance" calls your own `/anomaly/attendance` for the logged-in person's records only.
- **Proof to submit:** full-conversation tests, including the refusal of another person's salary.

Task 10 — Resume screening NLP

- **Inputs you already have:** job_descriptions (10), resumes (185, plain text), resume_job_labels (555), resume_bias_pairs (25).
- **Decisions already made:** resumes are plain text; there are no files to parse. Score from 0 to 100 = 60% required-skill coverage + 20% nice-to-have coverage + 20% experience fit, with a small synonym list. **Remove name, e-mail, gender cues and college tier**

before scoring. Return matched and missing skills. Targets: Spearman correlation with `human_suitability_score` of at least 0.6, and **all 25 bias pairs score identically**. Report accuracy by `layout_difficulty`.

- **Proof to submit:** correlation, bias-pair table, list of failures.

Weekly Milestone 2 (Tasks 6 to 10)

Build a “Smart Alerts” endpoint: HR asks for a month and gets one list of attendance and payroll flags sorted by importance, each with a reason. Decide how to rank them. Then answer in two lines each: why load a model at start-up, why the chatbot forwards the user’s token, which bias risks exist in resume screening, how to roll back a model in one minute. Save as `reports/weekly-2.md`.

Task 11 — Unified AI services API

- **Inputs you already have:** Section 6 contract.
- **Decisions already made:** routes exactly as the contract; one token check; one error format; `GET /ai/v1/health` reports each part (models loaded, chatbot reachable, backend reachable). Outgoing calls time out after 5 seconds so one slow part cannot freeze the rest. One Compose file starts everything.
- **Proof to submit:** `docs/ai-api-guide.md` with an example per endpoint and a smoke-test script.

Phase 3 review

Write `reports/phase-3-review.md`.

Phase 4 — Predictive analytics & first deployment

Task 12 — Attrition model v2

- **Inputs you already have:** the pipeline, the Task 3 split, sheet baseline_benchmarks.
- **Decisions already made:** Random Forest and XGBoost on the **same split and pipeline** as Task 3. Keep tuning small (about 30 minutes or 50 trials; Optuna is optional). Pick the threshold that catches at least 70% of leavers at the best precision. Check calibration

with a plot and use isotonic or Platt scaling. **Target: ROC-AUC at least 0.03 above the logistic baseline; if you cannot reach it, report the honest result.** Use the held-out test set once. Save as `attrition/2.0.0`.

- **Proof to submit:** comparison table, feature importance in plain words, calibration plot.

Task 13 — Salary / pay-band recommendation

- **Inputs you already have:** salary_training (2,934 hires), protected_attributes.
- **Decisions already made:** target `offered_annual_fixed`. Inputs: band minimum, mid and maximum, job level, prior experience, hire year or market index, work location, education, skills count. **Do not use gender, and also leave career_gap_months out as an input** (it can act as a proxy; you test it in Task 17). Baseline = median pay by role after dividing by the market index. Models: gradient boosting with quantile loss at 10%, 50% and 90%. Report MAE and MAPE against the baseline. The band should cover the true value **75 to 85%** of the time. If a role has fewer than 15 training rows, answer "not enough data".
- **Proof to submit:** metric table, coverage check, API example.

Task 14 — Workforce demand forecasting

- **Inputs you already have:** monthly_headcount_history (81 months), workforce_events, sample_forecast (output shape).
- **Decisions already made:** hold out the last 6 months. Compare a naive forecast, same-month-last-year and your model. Use **Prophet** with the hiring-drive and appraisal flags as extra inputs. If Prophet will not install, use statsmodels ETS or a regression with month dummies and say so in the README. Report MAPE and an 80% range. Forecast 6 months for the company and each department as JSON.
- **Proof to submit:** a forecast-versus-actual chart and a plain summary sentence.

Task 15 — Mandatory deployment: AI services

- **Inputs you already have:** the readiness checklist in Section 9.
- **Decisions already made:** the hosting platform is shared separately. Finish the readiness checklist now; on the day the platform details arrive, follow the "Platform day" steps. Ask the platform for at least 2 GB of memory if XGBoost, SHAP and spaCy are all loaded. Smoke tests cover login, anomaly scoring, attrition score, chatbot, resume score and forecast.

- **Questions answered:** *Do I wait for the platform?* No. The task is complete when the readiness checklist passes locally in production mode; the live link goes into your report when you deploy.
- **Proof to submit:** local production-mode run, smoke-test output and later the live link. Submit **Report 2** after Phase 4.

Weekly Milestone 3 (Tasks 11 to 15)

Build a “Model Report Card” endpoint: for each model it returns version, training date, key metrics and a plain-language summary. Then remove one model file on purpose and make the service report “degraded” in `/health` while the other features keep working. Answer: why check calibration, why forecasting needs a time-based split, why a salary band is more honest than one number. Save as `reports/weekly-3.md`.

Phase 4 review

Write `reports/phase-4-review.md`.

Phase 5 — Responsible AI & final release

Task 16 — Explainability layer

- **Inputs you already have:** your models, sheets sample_attrition_risk and attrition_scoring_2026.
- **Decisions already made:** SHAP for tree models (coefficients for logistic regression). `GET /ai/v1/explain/attrition/{employee_id}` returns `risk_score`, `top_factors` (feature, direction, plain text), one `sentence` and a `caution` line (“A support signal, not a decision”). The style of the text follows the columns `top_reason_1..3` of sample_attrition_risk. A department as a reason is worded “department has historically higher attrition” (context, not blame). Compare SHAP with LIME on 20 employees and note the differences. Ask one non-technical person to read 10 explanations and write down what was unclear.
- **Proof to submit:** explanations for the same 10 employees from both methods, and the readers’ notes.

Task 17 — AI bias & fairness review

- **Inputs you already have:** protected_attributes (join on `person_ref` and `record_type`), resume_protected_attrs, resume_bias_pairs, salary_training.

- **Decisions already made:** test gender and age band. Per group measure: share flagged high-risk, false-positive rate and false-negative rate; for salary, the average error and band coverage. **Guideline: the lowest group rate divided by the highest should be at least 0.80.** Look for proxies such as career gap. Try one or two fixes (drop a proxy, reweight, per-group threshold), measure again and write down the accuracy-versus-fairness trade-off. Write a model card (purpose, data, metrics, limits, who must not use it for what) for each model.
- **Questions answered:** the data is fake and the bias was built in on purpose. The **method** is what is being practised.
- **Proof to submit:** `reports/fairness-report.md` and three model cards in `docs/model-cards/`.

Task 18 — Model monitoring hooks

- **Inputs you already have:** training features (reference) and attrition_scoring_2026 (current).
- **Decisions already made:** drift per feature with PSI on 10 bins: below 0.10 stable, 0.10 to 0.25 watch, above 0.25 alert. Endpoints: `GET /ai/v1/metrics` (Prometheus text) with `cortexhr_requests_total`, `cortexhr_request_latency_seconds`, `cortexhr_prediction_high_risk_share` and `cortexhr_feature_psi{feature="..."}`; `GET /ai/v1/drift` (JSON). Logs use a shortened SHA-256 of the employee id, never names or pay. Prove an alert by shifting one feature in test data.
- **Proof to submit:** the endpoints, alert rules written in `docs/alerts.md`, and a simulated-drift test.

Task 19 — Unified AI assistant & simple chat screen

- **Inputs you already have:** all chatbot sheets, your services, the roles table in Section 5.
- **Decisions already made:** all 33 intents. Role rules: risk, payroll flags, salary bands and forecasts only for HR and ADMIN. Remember the last three turns so follow-ups such as “and for Finance?” work (see chatbot_dialogues). A “Why?” reply calls the explain endpoint. **Chat screen:** `chat.html` plus `chat-widget.js` that exposes `WorkforceChat.mount(element, {baseUrl, token})`; it works inside an iframe or a plain page. Create the design with any AI design tool, then review it with this rubric and record what you changed in `docs/design-notes.md`: text at least 16 px, colour contrast at least 4.5:1, works at 360 px width, has loading, empty and error states, shows “AI-generated, not a decision”, no jargon.

- **Proof to submit:** screenshots, `docs/design-notes.md`, conversation tests for an HR user and an EMPLOYEE user, and `docs/handoff-fullstack.md` (how to mount the widget, with a 10-line example).

Task 20 — Final AI handoff & consolidated deployment

- **Inputs you already have:** everything above, plus Section 9.
- **Decisions already made:** end-to-end stories: HR alert; employee chat (including the refusal of another person's salary); risk with "Why?"; planning (salary band plus forecast). Load test with Locust (50 users, 2 minutes); target a 95th-percentile latency under 2 seconds for scoring endpoints. Re-run the fairness and drift checks. Write the four handoff documents listed in Section 4. Deploy as in Section 9 and submit **Report 3**.
- **Final challenge (choose one and do it alone):** a weekly early-warning digest for each manager (only their team, with reasons, respecting role rules); a retraining command that promotes a new model only if it is better and fair; or the chatbot in one more language with a measured accuracy change.
- **Proof to submit:** test and load results, the four handoff files, tag `v1.0.0`, Report 3, a short description of your final challenge.

Weekly Milestone 4 (Tasks 16 to 20)

Write `reports/weekly-4.md` answering in plain words and without notes: how one raw payroll row becomes a flag with a reason; why a model with high accuracy can still be unfair or useless; how you would notice and respond to a model that is slowly going stale.

---

9. Deployment: be fully ready now, finish when the platform is shared

The hosting platform is announced separately when it is final. The name written in the brief is only a placeholder; ignore it until you receive the final platform. **No task is blocked** while you wait.

Readiness checklist (finish during Tasks 11 to 15)

#	Item	How to know it is done
1	`docker-compose.prod.yml` with the AI service (and Rasa if used)	Starts on a clean machine with one command

#	Item	How to know it is done
2	Slim image, runs as non-root, debug off	`ENV=production` set
3	Every setting comes from an environment variable	README table lists name, purpose and example
4	Models are inside the image or loaded at start from `models_store/`	`GET /ai/v1/models` lists them on a fresh start
5	`/ai/v1/health` checks models, chatbot and backend	All “ok”; a missing model shows “degraded”, not a crash
6	Logs go to the console; no names, pay or full employee ids in logs	Checked by reading a sample
7	`JWT_SECRET` read from the environment and at least 32 random characters in deployment	No secret in the repository
8	Outgoing calls have 5-second time-outs	Tested by stopping the backend
9	Smoke script `scripts/smoke.sh`	Logs in, scores attendance and payroll, gets an attrition score, chats, scores a resume, gets a forecast
10	README section “Deploy”	Steps below, filled with your own commands

Platform day (when the platform details are shared)

1. Create the service on the platform.
2. Set the variables from your README table: `ENV=production`, the platform’s `JWT_SECRET`, `BACKEND_BASE_URL` (the backend address you are given; **if you are not given one, run your mock backend as a second service and point to it**), `DEMO_TODAY=2026-09-30`.
3. Give the service at least 2 GB of memory if XGBoost, SHAP and spaCy are all loaded.
4. Deploy your image or compose file.
5. Open the health URL and check each item.
6. Run `scripts/smoke.sh` against the live URL.
7. Write the link and a screenshot in your report.

---

10. Reports, self-review and templates

Phase review (end of every phase, 10 lines at most)

`reports/phase-N-review.md`:

- **Done:** tasks finished, each with its pull-request link.
- **In progress:** what is half done.
- **Blocked:** anything stuck, with the assumption you made to keep going.
- **Next phase plan:** two lines.

Report 1 (end of Phase 2), Report 2 (end of Phase 4, with the deployment link), Report 3 (end of Phase 5)

Section	What to write
Tasks done vs planned	Table: task, planned, done (yes or no), pull-request link
What I learned	5 plain lines
What this unblocks	Which documents, formats or services other teams can now read, with links
Blockers	None, or the problem and the assumption you used
Model results or demo	A results table (metric, value, reference) and a screenshot or short recording
Deployment link	Reports 2 and 3 only (or “readiness complete, platform pending”)

Self-review before you merge any pull request

- [ ] All tests pass and the points in the task block are met.
- [ ] The service starts from a fresh clone with one command.
- [ ] README and `docs/` are updated; the Assumptions section is current.
- [ ] The test set was used only once (Task 12), and no leakage columns are used.
- [ ] No secrets, no real personal data.
- [ ] Every prediction shows a reason and a limits note.

- [ ] Evidence attached (results table, screenshot, test output).

Professional quality checklist (before Phase 4 and Phase 5 deployments)

- [ ] Clear README and API docs.
- [ ] Results explained in plain language.
- [ ] Realistic fake data.
- [ ] The chat screen (Task 19) looks clean: designed with an AI tool, then reviewed by you.
- [ ] A one-page summary a non-technical person can follow (`docs/summary.md`).

Security self-check (before Reports 2 and 3)

- [ ] Every endpoint checks the token and the role.
- [ ] An employee can only reach their own data through the chatbot and the scoring endpoints.
- [ ] Logs contain no names, pay or full ids.
- [ ] Input is validated; errors show no stack traces.
- [ ] Dependencies are pinned; no secrets in Git history.

Tool problems and the allowed fallback

Problem	Fix
Rasa will not install or train	Use the scikit-learn or spaCy fallback in Task 8
Prophet will not install	statsmodels ETS or a regression with month dummies
SHAP is slow on the full set	Explain a sample of 200 employees and say so
Docker image is too big	Use a slim base image, install only needed libraries, remove build tools
Sentence-transformers is too heavy	Use TF-IDF similarity instead

Questions people ask most

- **Whom do I send my documents to?** Nobody. Publish them in your repository as shown in Section 4.

- **Where do I submit my work?** Repository link, pull-request links and report files go through the same submission channel your programme already uses.
- **Which platform do I deploy to?** It is shared separately. Be ready as in Section 9.
- **Where do the chatbot's backend answers come from?** From the mock backend in Section 6, which reads workbook 01.
- **How do I make a login token for testing?** `scripts/make_token.py` in Section 5.
- **My score is near 1.0.** That is leakage. Check the columns that only exist after someone leaves.
- **A rule is missing from this guide.** Use the simplest sensible option, write it under Assumptions in your README, and continue.
- **Do I need the Python team's live backend?** No. The mock backend behaves the same way; switching is one variable.